EASV · SYSTEM INTEGRATION (E2026)

Bizcord — Messaging Microservice

Compulsory Assignment #1 · Weeks 36–40

A Slack/Discord-style business-communication platform, split into independently deployable microservices that talk over REST and RabbitMQ. This deck walks through all five tasks and the C#/.NET code behind them.

Benjamin Dam Christensen

What this assignment asked for

Bizcord is decomposed into **eight microservices**, each owning one bounded context. The assignment is to finalize the weekly labs from W36–W40 for **one** service — the Messaging service — and document the analysis & design in a `docs/` folder. Submission is a single repository link.

Task	Week	Deliverable in this repo
1. Analysis & Design	W36	`docs/architecture.md`, `docs/interactions.md`
2. Setup & Messaging	W37	Microservice in the monorepo + EasyNetQ message client
3. Inter-service communication	W38	Domain model, shared model, REST API
4. Deployment	W39	`Dockerfile` + `docker-compose.yml`
5. First feature & testing	W40	Channel membership feature + xUnit tests

> **Stack:** .NET 9 (ASP.NET Core minimal API) · RabbitMQ via EasyNetQ · in-memory repositories · Docker · xUnit

Task 1 — Analysis & Design

The platform splits into bounded contexts along *"one reason to change"*: auth changes for security reasons, messaging for product features, billing for pricing — so they become separate services. Two of them are **derived** services (Presence, Search): they mirror data owned elsewhere and consume events instead of owning the source of truth.

Service	Responsibility	Owns (data)
Identity & Access	accounts, SSO, profiles, sessions	users, credentials, sessions
Organisation & Membership	workspaces, members, roles, invites	organisations, memberships, roles
Messaging	channels, threads, DMs, reactions, read-state	channels, messages, reactions
Presence	online/away status, typing	presence state
Notifications	in-app, email, push delivery	preferences, delivery state
Files & Media	store/serve uploads	file metadata, blobs
Search	full-text index	search index *(derived)*
Billing & Subscriptions	plans, payments, invoices	subscriptions, payments, invoices

> **Boundary rule:** services never write to another service's store directly — they call its API or consume its events. Search is the clearest case: it keeps a read-only copy of messages, rebuilt from events, while Messaging stays the single source of truth.

Task 2 — Setup & the message client

The service lives in a monorepo (`apps/messaging-microservice`) and talks to RabbitMQ through a thin abstraction. The rest of the service depends on `IMessageClient` — not EasyNetQ directly — so the broker could be swapped without touching domain code.

src/MessagingService/Messaging/IMessageClient.cs

```
namespace MessagingService.Messaging;

/// <summary>
/// Thin publish/subscribe abstraction over the message broker. The rest of the
/// service depends on this instead of EasyNetQ directly, so the broker can be
/// swapped without touching the domain code.
/// </summary>
public interface IMessageClient
{
    Task PublishAsync<T>(T message) where T : class;
    Task SubscribeAsync<T>(string subscriptionId, Func<T, Task> handler) where T : class;
}
```

src/MessagingService/Messaging/MessageClient.cs

```
using EasyNetQ;

namespace MessagingService.Messaging;

/// <summary>
/// EasyNetQ-backed implementation of <see cref="IMessageClient"/>.
/// </summary>
public class MessageClient : IMessageClient
{
    private readonly IBus _bus;

    public MessageClient(IBus bus) => _bus = bus;

    public Task PublishAsync<T>(T message) where T : class
        => _bus.PubSub.PublishAsync(message);

    public Task SubscribeAsync<T>(string subscriptionId, Func<T, Task> handler) where T : class
        => _bus.PubSub.SubscribeAsync(subscriptionId, handler);
}
```

Wiring in `Program.cs` — EasyNetQ reads the connection string, and the repositories/message client are registered for dependency injection:

src/MessagingService/Program.cs (lines 19–27)

```
// RabbitMQ (EasyNetQ) + the message client wrapper.
var rabbitMq = builder.Configuration.GetConnectionString("RabbitMQ") ?? "host=localhost";
builder.Services.AddEasyNetQ(rabbitMq);
builder.Services.AddSingleton<IMessageClient, MessageClient>();

// In-memory persistence (swap for a real store later).
builder.Services.AddSingleton<IChannelRepository, InMemoryChannelRepository>();
builder.Services.AddSingleton<IMessageRepository, InMemoryMessageRepository>();
builder.Services.AddSingleton<IChannelMemberRepository, InMemoryChannelMemberRepository>();
```

Task 3 — Domain model (internal entities)

The domain entities are the **internal** truth — richer than what other services need. `ChannelMember`, for example, keeps the role, per-channel permission overrides *and* an audit log of every role change.

src/MessagingService/Domain/Channel.cs

```
namespace MessagingService.Domain;

/// <summary>
/// A channel (or direct-message thread) in a workspace. This is the full internal
/// entity — the shared model (Contracts) exposes only a subset of these fields.
/// </summary>
public class Channel
{
    public Guid Id { get; init; } = Guid.NewGuid();
    public Guid WorkspaceId { get; init; }
    public string Name { get; set; } = "";
    public string Description { get; set; } = "";
    public bool IsDirectMessage { get; init; }
    public DateTimeOffset CreatedAt { get; init; } = DateTimeOffset.UtcNow;
}
```

src/MessagingService/Domain/Message.cs

```
namespace MessagingService.Domain;

public class Message
{
    public Guid Id { get; init; } = Guid.NewGuid();
    public Guid ChannelId { get; init; }
    public Guid AuthorId { get; init; }
    public string Content { get; set; } = "";
    public DateTimeOffset CreatedAt { get; init; } = DateTimeOffset.UtcNow;
    public DateTimeOffset? EditedAt { get; set; }
}
```

src/MessagingService/Domain/ChannelMember.cs (excerpt)

```
public class ChannelMember
{
    public Guid Id { get; init; } = Guid.NewGuid();
    public Guid ChannelId { get; init; }
    public Guid UserId { get; init; }
    public ChannelRole Role { get; private set; }

    // Internal: per-channel overrides on top of the role's defaults.
    public List<PermissionOverride> Overrides { get; } = [];

    // Internal: audit log of every role change.
    public List<RoleChange> RoleHistory { get; } = [];

    public ChannelMember(Guid channelId, Guid userId, ChannelRole role = ChannelRole.Member)
    {
        ChannelId = channelId;
        UserId = userId;
        Role = role;
    }

    public void ChangeRole(ChannelRole newRole, Guid changedBy)
    {
        if (newRole == Role)
        {
            return;
        }

        RoleHistory.Add(new RoleChange(Role, newRole, changedBy, DateTimeOffset.UtcNow));
        Role = newRole;
    }
```

Task 3 — Shared model (the contract others integrate with)

The `Contracts/` records are the **narrow** surface other services see. A `ChannelMember` has overrides and a role history internally, but the API only exposes `userId`, `roleType` and the *resolved* permissions.

src/MessagingService/Contracts/ChannelMemberDto.cs

```
namespace MessagingService.Contracts;

/// <summary>
/// Shared model for a channel membership. Other services only need to know who is
/// in a channel, their role, and what they can do — not the override list or the
/// audit log the domain keeps internally.
/// </summary>
public record ChannelMemberDto(Guid UserId, ChannelRoleType RoleType, ChannelPermissions Permissions);

public enum ChannelRoleType
{
    Member,
    Moderator,
    Admin
}

public record ChannelPermissions(bool CanPost, bool CanInvite, bool CanManageChannels);

public record CreateChannelMemberRequest(Guid UserId, ChannelRoleType RoleType);

public record UpdateMemberRoleRequest(ChannelRoleType RoleType, Guid ChangedBy);
```

src/MessagingService/Contracts/ChannelDto.cs

```
namespace MessagingService.Contracts;

/// <summary>
/// Shared model for a channel — only the fields other services need to integrate
/// with. Internal details (workspace id, DM flag) stay in the domain entity.
/// </summary>
public record ChannelDto(Guid Id, string Name, string Description, DateTimeOffset CreatedAt);

public record CreateChannelRequest(string Name, string Description);
```

src/MessagingService/Contracts/MessageDto.cs

```
namespace MessagingService.Contracts;

public record MessageDto(Guid Id, Guid ChannelId, Guid AuthorId, string Content, DateTimeOffset CreatedAt);

public record CreateMessageRequest(Guid AuthorId, string Content);
```

The mapping from domain → shared model lives on the entity itself, so the resolution logic (role defaults + overrides) stays in one place:

src/MessagingService/Domain/ChannelMember.cs — EffectivePermissions + ToDto

```
    public ChannelPermissions EffectivePermissions()
    {
        var permissions = Role switch
        {
            ChannelRole.Admin => new ChannelPermissions(true, true, true),
            ChannelRole.Moderator => new ChannelPermissions(true, true, false),
            _ => new ChannelPermissions(true, false, false),
        };

        foreach (var o in Overrides)
        {
            permissions = o.Permission switch
            {
                nameof(ChannelPermissions.CanPost) => permissions with { CanPost = o.Allowed },
                nameof(ChannelPermissions.CanInvite) => permissions with { CanInvite = o.Allowed },
                nameof(ChannelPermissions.CanManageChannels) => permissions with { CanManageChannels = o.Allowed },
                _ => permissions,
            };
        }

        return permissions;
    }
```

```
    public ChannelMemberDto ToDto()
    {
        var roleType = Role switch
        {
            ChannelRole.Admin => ChannelRoleType.Admin,
            ChannelRole.Moderator => ChannelRoleType.Moderator,
            _ => ChannelRoleType.Member,
        };

        return new ChannelMemberDto(UserId, roleType, EffectivePermissions());
    }
```

Task 3 — REST API (sync) + events (async)

The minimal API exposes channels, messages and members. The interesting part is the `POST` on messages: it **persists + replies synchronously**, then **publishes a `MessageCreated` event** so Notifications/Search can react without the caller waiting.

src/MessagingService/Program.cs — channels & messages endpoints

```
// --- Channels ---

app.MapGet("/channels", async (IChannelRepository channels) =>
{
    var all = await channels.GetAllAsync();
    return Results.Ok(all.Select(c => new ChannelDto(c.Id, c.Name, c.Description, c.CreatedAt)));
});

app.MapGet("/channels/{id:guid}", async (Guid id, IChannelRepository channels) =>
{
    var channel = await channels.GetByIdAsync(id);
    return channel is null
        ? Results.NotFound()
        : Results.Ok(new ChannelDto(channel.Id, channel.Name, channel.Description, channel.CreatedAt));
});

app.MapPost("/channels", async (CreateChannelRequest request, IChannelRepository channels) =>
{
    var channel = new Channel { Name = request.Name, Description = request.Description };
    await channels.AddAsync(channel);
    return Results.Created($"/channels/{channel.Id}", new ChannelDto(channel.Id, channel.Name, channel.Description, channel.CreatedAt));
});

// --- Messages ---

app.MapGet("/channels/{channelId:guid}/messages", async (Guid channelId, IMessageRepository messages) =>
{
    var result = await messages.GetByChannelAsync(channelId);
    return Results.Ok(result.Select(m => new MessageDto(m.Id, m.ChannelId, m.AuthorId, m.Content, m.CreatedAt)));
});

app.MapPost("/channels/{channelId:guid}/messages", async (
    Guid channelId,
    CreateMessageRequest request,
    IMessageRepository messages,
    IMessageClient client) =>
{
    var message = new Message { ChannelId = channelId, AuthorId = request.AuthorId, Content = request.Content };
    await messages.AddAsync(message);

    // REST complements messaging: persist + reply synchronously, but also publish
    // an event so Notifications/Search can react without the caller waiting.
    await client.PublishAsync(new MessageCreated(message.Id, message.ChannelId, message.AuthorId, message.Content, message.CreatedAt));

    return Results.Created(
        $"/channels/{channelId}/messages/{message.Id}",
        new MessageDto(message.Id, message.ChannelId, message.AuthorId, message.Content, message.CreatedAt));
});
```

src/MessagingService/Messages/MessageCreated.cs

```
namespace MessagingService.Messages;

/// <summary>
/// Published when a user posts a message. Downstream services (Notifications,
/// Search) subscribe to this so the sender never waits on them.
/// </summary>
public record MessageCreated(
    Guid MessageId,
    Guid ChannelId,
    Guid AuthorId,
    string Content,
    DateTimeOffset CreatedAt);
```

> **Why both?** REST (request/response) when the caller needs an answer now; events (fire-and-forget) when the work can happen later. This keeps the core Messaging path fast — it publishes and moves on instead of waiting for every downstream service.

Task 4 — Deployment

A multi-stage `Dockerfile` builds in the SDK image and runs in the tiny `aspnet` runtime image. `docker-compose` then wires RabbitMQ and the service together with a healthcheck so the service only starts once the broker is up.

Dockerfile

```
# Build stage
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src

COPY src/MessagingService/MessagingService.csproj src/MessagingService/
RUN dotnet restore src/MessagingService/MessagingService.csproj

COPY src/MessagingService/ src/MessagingService/
RUN dotnet publish src/MessagingService/MessagingService.csproj -c Release -o /app/publish

# Runtime stage
FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
EXPOSE 8080
ENTRYPOINT ["dotnet", "MessagingService.dll"]
```

docker-compose.yml

```
# Bizcord messaging microservice + RabbitMQ.
# Task 4 — Deployment: Dockerfile + docker-compose.
services:
  rabbitmq:
    image: rabbitmq:3-management
    ports:
      - "5672:5672"    # AMQP
      - "15672:15672"  # management UI (guest/guest)
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 5s
      timeout: 5s
      retries: 20

  messaging-service:
    build: .
    ports:
      - "8080:8080"
    depends_on:
      rabbitmq:
        condition: service_healthy
    environment:
      # EasyNetQ connection string — the broker is reachable by service name.
      ConnectionStrings__RabbitMQ: "host=rabbitmq"
```

Task 5 — First feature & testing

The “first feature” is channel membership. The tests cover the domain logic the REST API and shared model build on: role changes, the audit log, effective permissions, and the narrow DTO. **8 tests pass** (`dotnet test`).

tests/MessagingService.Tests/ChannelMemberTests.cs (excerpt)

```
    [Fact]
    public void ChangeRole_UpdatesRole_AndRecordsAuditEntry()
    {
        var member = new ChannelMember(_channelId, _userId, ChannelRole.Member);

        member.ChangeRole(ChannelRole.Moderator, Guid.NewGuid());

        Assert.Equal(ChannelRole.Moderator, member.Role);
        var entry = Assert.Single(member.RoleHistory);
        Assert.Equal(ChannelRole.Member, entry.From);
        Assert.Equal(ChannelRole.Moderator, entry.To);
    }

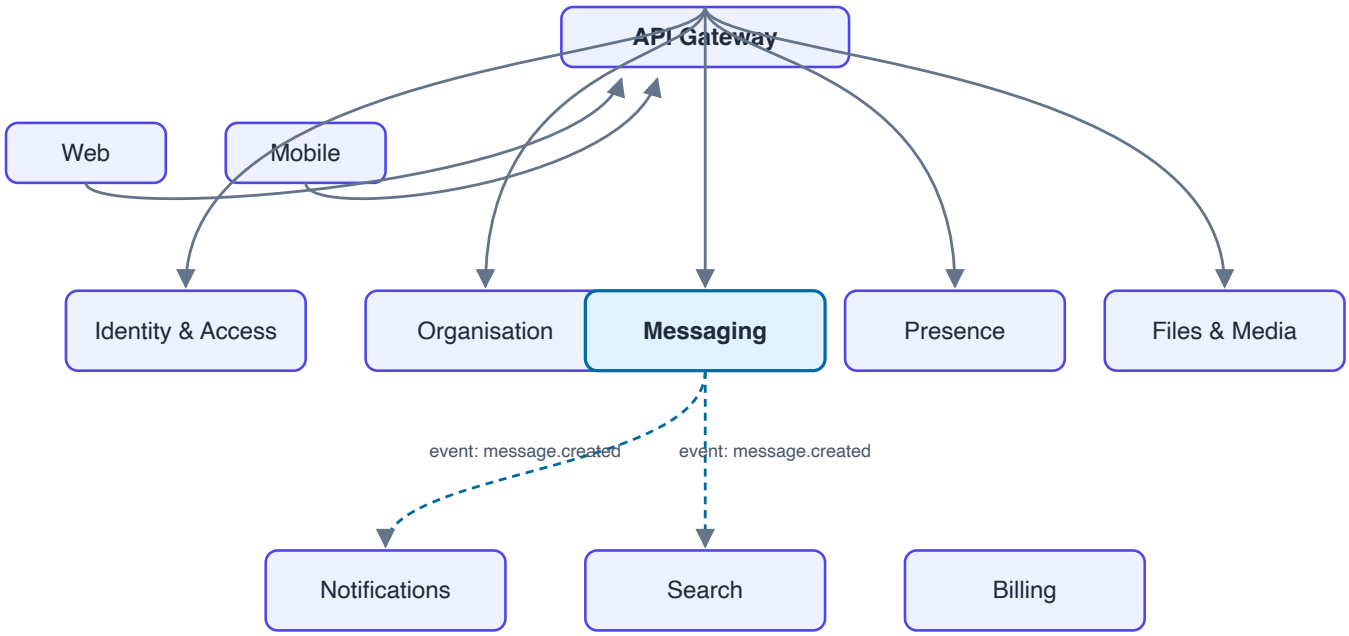
    [Fact]
    public void ChangeRole_ToSameRole_IsNoop()
    {
        var member = new ChannelMember(_channelId, _userId, ChannelRole.Member);

        member.ChangeRole(ChannelRole.Member, Guid.NewGuid());

        Assert.Empty(member.RoleHistory);
        Assert.Equal(ChannelRole.Member, member.Role);
    }
```

> **Verification:** `dotnet build` → 0 errors, 0 warnings · `dotnet test` → 8 passed · service runs via `docker compose up` (RabbitMQ + messaging-service).

The architecture at a glance



solid = REST (sync) · dashed = event (async, over RabbitMQ)

> **Sync vs async, once more:** the gateway calls services over REST for request/response; Messaging fans a `message.created` event out to Notifications and Search so the sender is never blocked.

Key takeaways

- **Bound by "one reason to change"** — each service owns one bounded context and its data.
- **Domain model ≠ shared model** — entities stay rich internally; other services only see narrow DTOs.
- **REST for answers, events for side-effects** — persist + reply, then publish; never block the hot path.
- **Abstractions over infrastructure** — `IMessageClient` hides the broker; repositories hide storage.
- **Ship it reproducibly** — multi-stage Docker build + compose with healthchecks.
- **Test the domain** — 8 xUnit tests prove role changes, audit logging and permission resolution.

> Repo: `github.com/BenjaminDamC/bizcord` · service: `apps/messaging-microservice` · docs: `docs/architecture.md` + `docs/interactions.md`